Iterative LLM-Based NAS with Feedback Memory

Xiaojie Gu, Dmitry Ignatov, Radu Timofte
Computer Vision Lab, CAIDAS & IFI, University of Würzburg, Germany

Abstract

Neural Architecture Search (NAS) automates network design but typically demands substantial computational resources. We propose a closed-loop pipeline that leverages large language models (LLMs) to iteratively generate, evaluate, and refine convolutional neural network architectures for image classification. Central to our approach is a historical feedback memory *mechanism inspired by Markov chains: a sliding window of the most recent improvement attempts is fed back to the LLM, enabling it to learn from past successes and failures without additional training. Using one-epoch proxy accuracy on CIFAR-10 [12] as a fast architecture ranking signal, we evaluate the pipeline with four instruction-tuned LLMs of different sizes. Qwen2.5-7B-Instruct improves generated architectures from 38.0% to 59.3% over 77 successful evaluations within 100 iterations (Spearman* $\rho = 0.67$, $p < 10^{-10}$*), Qwen2.5-Coder-7B-Instruct improves from 46.4% to 63.6% over 56 successful evaluations (Spearman* $\rho = 0.51$, $p < 10^{-4}$*), DeepSeek-Coder-6.7B-Instruct improves from 37.7% to 63.0% over 39 successful evaluations (Spearman* $\rho = 0.56$, $p < 10^{-3}$*), while the smaller Qwen3-4B-Instruct improves from 10.0% to 37.9% over 41 successful evaluations (Spearman* $\rho = 0.79$, $p < 10^{-9}$*). All models exhibit statistically significant upward trends. Compared to prior iterative LLM optimization methods that rely on global elite retention (discarding failure trajectories) [19, 25], require complex population maintenance [2], or operate in constrained discrete search spaces [27], our bounded Markovian memory explicitly models sequential code execution failures in an open code space. This structured causal learning from recent mistakes is a critical requirement for neural architecture generation that distinguishes our approach from existing work. These results also establish that even small LLMs can serve as effective architecture designers when equipped with iterative feedback and historical memory, offering a lightweight and accessible paradigm for LLM-driven NAS.*

1. Introduction

Neural Architecture Search (NAS) has emerged as a powerful paradigm for automating the design of deep neural networks, achieving competitive or superior performance to hand-crafted architectures across diverse tasks [14, 18, 28]. However, conventional NAS methods—whether based on reinforcement learning [28], evolutionary algorithms [18], or differentiable relaxations [14]—remain computationally expensive, often requiring hundreds or thousands of GPU hours to search a single architecture. This cost has motivated training-free approaches that use proxy metrics to rank architectures without full training [13, 15], yet these methods still operate within predefined, discrete search spaces.

Recent advances in large language models (LLMs) have opened a fundamentally different avenue: using LLMs as architecture generators that produce executable neural network code directly [9, 11, 24]. Unlike traditional NAS, LLM-based approaches operate in the space of *programs* rather than fixed cell-based encodings, enabling more flexible and expressive architecture design. Prior work within the NNGPT framework [11] and related efforts [7, 16, 22] have demonstrated that LLMs can generate functional vision models, but predominantly through single-shot or few-shot generation without iterative self-improvement.

A critical limitation of single-shot LLM generation is that it treats architecture design as a one-pass prediction, discarding the evaluation signal entirely. In contrast, human engineers iterate: they build, test, analyze failures, and refine their designs based on accumulated experience. This observation motivates our central question: *Can a small LLM iteratively improve neural network architectures by learning from a structured history of its own attempts?*

We propose a closed-loop pipeline comprising three components: (1) a **Code Generator** that prompts an instruction-tuned LLM to produce PyTorch model implementations, (2) an **Evaluator** that trains each generated model for a single epoch on CIFAR-10 as a fast proxy for architecture quality, and (3) a **Prompt Improver** that analyzes evaluation results alongside a sliding window of recent iteration history—a mechanism we term *historical feedback memory*—to produce targeted improvement sug-

gestions for the next iteration. This design draws on the Markov property: the improvement decision at each step depends on the current best architecture and a bounded window of recent transitions, rather than the full trajectory.

Our main contributions are:

- A closed-loop, iterative NAS pipeline driven by LLMs that progressively discovers better architectures through code generation, evaluation, and prompt refinement.
- A *historical feedback memory* mechanism that maintains a sliding window of past improvement attempts, enabling the LLM to mitigate repeating failed strategies and build on successful ones.
- An empirical demonstration across four LLMs of different sizes (4B to 7B parameters) showing that the pipeline consistently improves architecture quality—from 38.0% to 59.3% (Qwen2.5-7B), 46.4% to 63.6% (Qwen2.5-Coder-7B), 37.7% to 63.0% (DeepSeek-Coder-6.7B), and 10.0% to 37.9% (Qwen3-4B)—with strong positive trends (Spearman $\rho \geq 0.51$, $p < 10^{-3}$).

2. Related Work

Neural Architecture Search. NAS methods aim to automate the design of neural network architectures. Early approaches employed reinforcement learning [28] and evolutionary algorithms [18] to explore architecture search spaces, but required thousands of GPU hours per search. Parameter-sharing methods such as ENAS [17] and differentiable approaches like DARTS [14] substantially reduced search cost by amortizing evaluation across architectures. More recently, *zero-shot* or *training-free* NAS methods [13, 15] bypass training entirely by scoring architectures at initialization using proxy metrics such as activation overlap [15] or gradient-based indicators. While effective within their respective search spaces, these methods remain constrained to predefined, discrete architecture parameterizations (e.g., cell-based structures) and cannot generate truly novel architectural patterns.

LLMs for Code Generation and AutoML. Large language models have demonstrated strong capabilities in code generation [3], and their application to machine learning automation is an active research area. The NNGPT framework [11] pioneered the use of LLMs as neural network generators, showing that language models can produce functional vision models from textual prompts. Subsequent work explored fractal-inspired architectures [16], few-shot prompting strategies [24], novel architecture creativity [9], and non-standard channel priors [22]. LLMs have also been applied to hyperparameter optimization [10], data transformation design [21], and collaborative vision-language pipelines [5, 8, 20]. These efforts are supported by curated datasets of neural network architectures [6, 23] and deployment pipelines [4]. However, most existing approaches treat LLM-based architecture generation as a single-step process, without leveraging iterative refinement from evaluation feedback. Our work extends this line by introducing a closed-loop pipeline with historical memory that enables progressive architecture improvement.

Iterative LLM Optimization. Recent work has explored using LLMs as iterative optimizers that progressively improve solutions by learning from historical attempts. OPRO [25] demonstrated that LLMs can optimize text prompts and discrete solutions by conditioning on sorted history of past candidates and scores. FunSearch [19] extended this to program synthesis for mathematical discovery using island-based evolutionary search. EvoPrompting [2] applied evolutionary prompting with LLMs as mutation operators for code-level neural architecture search. ReEvo [26] introduced dual-layer reflective memory for LLM-driven heuristic algorithm design. More directly related to NAS, LLMO [27] used LLMs as combinatorial optimizers within the predefined cell-based search space of NAS-Bench-201, replacing traditional meta-heuristics. However, existing methods face key limitations: reliance on global elite histories that discard crucial failure trajectories [19, 25], the computational overhead of maintaining evolutionary populations [2], operation in constrained discrete search spaces [27], or targeting domains without structural code constraints [26]. Our work introduces bounded Markovian memory specifically designed for iterative neural architecture code generation in open search spaces, with explicit failure modeling enabling effectiveness on small frozen LLMs.

3. Methodology

Inspired by recent advancements in the application of LLMs across various domains [5, 8, 10, 20] and prior architectural synthesis experiments within the NNGPT framework [7, 9, 10, 16, 21, 22, 24], and leveraging the existing LEMUR dataset of a broad range of high-capacity and edge-optimized models [4, 6, 23], we developed an iterative NAS pipeline based on the NNGPT framework [11]. The pipeline consists of three core modules operating in a closed loop, as illustrated in Figure 1.

3.1. Pipeline Overview

At each iteration t, the pipeline executes the following cycle: (1) the Code Generator produces a candidate architecture $\mathcal{A}_t$ as executable PyTorch code; (2) the generated code is validated and, if structurally correct, trained and evaluated by the Evaluator to obtain a proxy accuracy a_t; (3) the Prompt Improver analyzes the result in the context of the current best architecture and recent iteration history, producing improvement suggestions s_t for the next iteration.

The best-performing architecture $\mathcal{A}^* = \arg\max_{i \leq t} a_i$ is maintained as the reference implementation throughout the search.

3.2. Code Generator

The Code Generator uses a pre-trained, instruction-tuned LLM to produce a complete PyTorch model class implementing `nn.Module`. The LLM receives a fixed prompt template comprising: (i) a role description positioning the model as a "visionary deep learning architect," (ii) the task specification (CIFAR-10 image classification without pre-trained weights), (iii) the current best implementation $\mathcal{A}^*$ as reference code, and (iv) the improvement suggestions s_{t-1} from the previous iteration. The generated code must define a `Net(nn.Module)` class with standard `__init__` and `forward` methods. Generation uses temperature $\tau = 0.7$ and nucleus sampling ($p = 0.9$) to balance architectural diversity with code coherence.

3.3. Evaluator

Generated code undergoes a two-stage evaluation process.

Quick validation. The model is instantiated and a forward pass is performed with a dummy input tensor of shape $2 \times 3 \times 32 \times 32$. The output shape is verified to match $B \times 10$ (for batch size B and 10 CIFAR-10 classes). Models failing this check are discarded with an error message forwarded to the Prompt Improver.

Proxy training. Models passing validation are trained for **one epoch** on CIFAR-10 using SGD (momentum 0.9, weight decay 5×10^{-4}), an initial learning rate of 0.01 with cosine annealing, and a batch size of 128. Standard data augmentation is applied: random crop with 4-pixel padding and random horizontal flip. Top-1 test accuracy on the 10,000-image test set serves as the proxy metric. Training is executed in an isolated subprocess with a 30-minute timeout. One-epoch accuracy provides a fast, informative signal for ranking architectures [15], enabling rapid iteration without the cost of full convergence training.

3.4. Prompt Improver with Historical Feedback Memory

The Prompt Improver is the central component enabling iterative learning. After each evaluation, it receives three inputs: (1) the best code $\mathcal{A}^*$ and its accuracy a^*, (2) the current iteration's code $\mathcal{A}_t$ and its evaluation outcome (accuracy value or error message), and (3) a *historical feedback memory* $\mathcal{H}_t$ containing the last K=5 improvement attempts.

Each history entry $h_i \in \mathcal{H}_t$ is a triple recording: the identified problem, the suggested improvement, and the resulting outcome (accuracy achieved or error encountered). This sliding-window design draws on the Markov property: the improvement decision at step t depends on the current state (best architecture plus recent history window) rather than the complete trajectory. The bounded window prevents context overflow while providing sufficient signal for the LLM to identify recurring failure patterns.

Markov Property Formalization. Let $\mathcal{A}_t$ denote the architecture generated at step t, a_t its evaluation outcome (accuracy or error), and s_t the improvement suggestions. Our historical feedback memory $\mathcal{H}_t^{(K)}$ maintains exactly the most recent K=5 improvement attempts:

$$\mathcal{H}_t^{(K)} = \{(s_{t-K}, a_{t-K}), \ldots, (s_{t-1}, a_{t-1})\}. \tag{1}$$

The improvement suggestion generation satisfies the **K-order Markov property**:

$$P(s_t \mid \mathcal{A}^*, \mathcal{H}_t) = P(s_t \mid \mathcal{A}^*, \mathcal{H}_t^{(K)}), \tag{2}$$

meaning the next suggestion depends only on the current best architecture and the bounded recent history, not the complete trajectory. This design ensures **constant context size** and avoids the context overflow problem faced by unbounded history approaches like OPRO [25].

Each history entry (s_i, a_i) is structured as a **diagnostic triple**:

$$s_i = (\text{problem}_i, \text{suggestion}_i, \text{outcome}_i), \tag{3}$$

where problem_i identifies the architectural deficiency, suggestion_i proposes concrete code modifications, and outcome_i records whether the suggestion succeeded (accuracy gain) or failed (error type). This structured format enables the LLM to learn causal patterns between design decisions and outcomes, unlike scalar-score histories in prior work [19, 25].

The Prompt Improver produces a structured response containing: (a) a **reason** diagnosing the current result, (b) an **inspiration** drawing on cross-disciplinary insights, and (c) concrete **improvement suggestions** for the next iteration. These suggestions, together with the best code, form the input to the Code Generator in the subsequent iteration, closing the feedback loop.

4. Experiments

4.1. Experimental Setup

Dataset. We use CIFAR-10 [12], comprising 60,000 32×32 color images across 10 classes (50,000 training, 10,000 test). Standard augmentation (random crop with 4-pixel padding, random horizontal flip, normalization) is applied during training, inspired by [1].

Algorithm 1: Iterative LLM-Driven NAS Pipeline

Input: LLM $\mathcal{L}$, max iterations T, history window K

$\mathcal{A}^* \leftarrow \emptyset$; $a^* \leftarrow 0$; $\mathcal{H} \leftarrow \emptyset$; $s_0 \leftarrow \emptyset$;
for $t \leftarrow 1$ **to** T **do**
  $\mathcal{A}_t \leftarrow \text{Generate}(\mathcal{L}, \mathcal{A}^*, s_{t-1})$;
  if *Validate*$(\mathcal{A}_t)$ *succeeds* **then**
    $a_t \leftarrow \text{TrainAndEvaluate}(\mathcal{A}_t)$;
  else
    $a_t \leftarrow$ error message;
  if $a_t > a^*$ **then**
    $\mathcal{A}^* \leftarrow \mathcal{A}_t$; $a^* \leftarrow a_t$;
  Append (s_{t-1}, a_t) to $\mathcal{H}$; keep last K entries;
  $s_t \leftarrow \text{Improve}(\mathcal{L}, \mathcal{A}^*, \mathcal{A}_t, a_t, \mathcal{H})$;
return $\mathcal{A}^*$, a^*

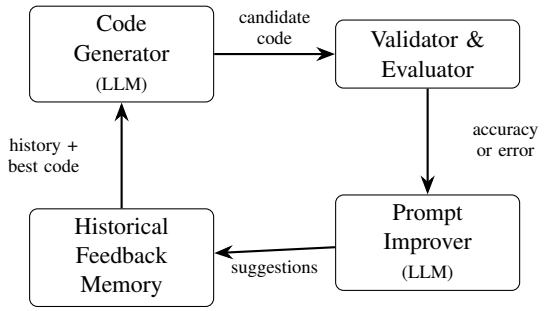

Figure 1. Overview of the iterative NAS pipeline. The Code Generator produces candidate architectures as executable PyTorch code. The Evaluator validates and trains them using one-epoch proxy evaluation. The Prompt Improver analyzes results with historical feedback memory to generate targeted improvement suggestions for the next iteration.

Language models. To evaluate the pipeline across model sizes and specializations, we run separate experiments with four instruction-tuned LLMs: **Qwen2.5-7B-Instruct**, **Qwen3-4B-Instruct**, **Qwen2.5-Coder-7B-Instruct**, and **DeepSeek-Coder-6.7B-Instruct**. Both generalist and code-specialized models use the same generation parameters: temperature 0.7, nucleus sampling $p = 0.9$, maximum 2,048 new tokens. A fixed seed with incremental call counter ensures reproducible but diverse generation across iterations. Each model is run for 100 iterations.

Training protocol. Each candidate architecture is trained for one epoch with SGD (momentum 0.9, weight decay 5×10^{-4}), learning rate 0.01 with cosine annealing, and batch size 128. A fixed random seed (43) and deterministic CUDA operations ensure reproducibility. Training runs in isolated subprocesses with a 30-minute timeout. Fine-tuning of LLMs and training of computer vision models are performed on NVIDIA GeForce RTX 4090 24G GPU.

Baseline. We compare against **single-shot generation**: the accuracy of the first successfully generated and evaluated architecture (iteration 1), representing what the LLM produces from its pre-trained knowledge alone without any feedback signal.

4.2. Evaluation Metrics

We report top-1 test accuracy after one-epoch training as the primary architecture quality metric. To characterize search dynamics, we compute: Spearman rank correlation ρ between iteration index and accuracy (trend strength), linear regression slope (improvement rate per 10 iterations), coefficient of determination R^2, and descriptive statistics (mean $\pm$ SD) for early and late evaluation blocks.

4.3. Main Results

Table 1 compares the key results for all four LLMs over 100 iterations each. All models exhibit statistically significant upward trends in architecture quality, confirming that the iterative pipeline with historical feedback memory consistently drives improvement regardless of model size or domain specialization.

Qwen2.5-7B-Instruct. The larger generalist model achieved 77 successful evaluations out of 100 iterations (77% success rate). One-epoch proxy accuracy improved from 38.0% (single-shot baseline) to a peak of 59.3% at the 68th successful evaluation, with the last 5 evaluations averaging $57.1\% \pm 3.2\%$. The Spearman correlation between evaluation index and accuracy is $\rho = 0.670$ ($p < 10^{-10}$), and the linear regression slope is $+1.2$ percentage points per 10 evaluations ($R^2 = 0.203$).

Qwen3-4B-Instruct. The smaller model achieved 41 successful evaluations out of 100 iterations (41% success rate). One-epoch proxy accuracy improved from 10.0% to a peak of 37.9% at the 31st successful evaluation, with the last 5 evaluations averaging $37.1\% \pm 1.6\%$. Despite the lower absolute accuracy, the Spearman correlation is higher at $\rho = 0.787$ ($p < 10^{-9}$), with a steeper linear slope of $+6.5$ percentage points per 10 evaluations ($R^2 = 0.557$), reflecting the larger room for improvement from a lower starting point.

Qwen2.5-Coder-7B-Instruct. The code-specialized 7B model achieved 56 successful evaluations out of 100 iterations (56% success rate). One-epoch proxy accuracy improved from 46.4% to a peak of 63.6% at the 36th successful evaluation, reaching the highest final accuracy among all models. The last 5 evaluations averaged $55.9\% \pm 12.5\%$,

exhibiting some variance. The Spearman correlation is $\rho = 0.513$ ($p < 10^{-4}$), with a linear slope of $+2.8$ percentage points per 10 evaluations ($R^2 = 0.192$).

DeepSeek-Coder-6.7B-Instruct. The code-specialized 6.7B model achieved 39 successful evaluations out of 100 iterations (39% success rate). One-epoch proxy accuracy improved from 37.7% to a peak of 63.0% at the 38th successful evaluation, with the last 5 evaluations averaging $59.0\% \pm 2.6\%$. The Spearman correlation is $\rho = 0.556$ ($p < 10^{-3}$), and the linear slope is $+5.5$ percentage points per 10 evaluations ($R^2 = 0.522$).

Figure 2 shows the accuracy trajectories for all models. Strong baselines and steady climbing are visible for the 7B models, while Qwen3-4B displays a rapid-improvement phase from a near-random baseline. All curves display characteristic convergence behavior with diminishing returns in later evaluations.

4.4. Failure Analysis

The models exhibit markedly different failure profiles. Qwen2.5-7B-Instruct achieved a 77% success rate (77 of 100 iterations), with all 23 failures attributable to **code validation errors**—tensor shape mismatches, channel dimension conflicts, and `AttributeErrors` from undefined modules. Notably, the 7B model produced zero code extraction errors, indicating reliable generation of syntactically valid Python.

Qwen3-4B-Instruct achieved a lower 41% success rate (41 of 100 iterations), with failures split between **code validation errors** (48 of 100) and **code extraction errors** (11 of 100). The extraction failures—unclosed parentheses, unterminated string literals, and syntax errors—suggest that the smaller model more frequently produces malformed code that cannot even be parsed.

The code-specialized models, Qwen2.5-Coder-7B and DeepSeek-Coder-6.7B, achieved success rates of 56% and 39%, respectively. While their syntactical correctness was generally high, they often explored more complex architectural patterns, leading to a higher frequency of validation errors related to tensor dimensions and invalid operations.

The dominant validation failure mode for all models is tensor shape mismatches between layers, followed by `AttributeErrors` from hallucinated layer names. Despite the different success rates, all models produce architectures that exhibit consistent improvement across their respective successful evaluations, confirming the pipeline's resilience to high failure rates.

5. Discussion

Effectiveness of iterative refinement. All models exhibit statistically significant positive correlations between evaluation index and accuracy ($\rho \in [0.51, 0.79]$), confirming that historical feedback memory enables systematic architectural improvement across different LLM sizes and specializations. Access to the last $K{=}5$ improvement attempts provides the LLM with sufficient context to identify recurring failure patterns and avoid previously unsuccessful strategies. All accuracy curves display characteristic increasing behavior—rapid early gains followed by gradual increasing—mirroring patterns in traditional optimization and suggesting that the LLM-driven search exhibits meaningful optimization dynamics rather than random exploration.

Effect of model size and specialization. The comparison between the models reveals a clear trade-off. The generalist Qwen2.5-7B generates structurally valid code most reliably (77% success rate). However, the smaller Qwen3-4B exhibits a stronger relative improvement trend ($\rho = 0.79$, slope $5.4\times$ steeper than Qwen2.5-7B), reflecting greater room for improvement from its lower starting point. Meanwhile, the code-specialized models (Qwen2.5-Coder and DeepSeek-Coder) reach the highest peak accuracies (63.6% and 63.0%) from their higher initial baselines, suggesting that domain specialization provides better architectural intuition, albeit with lower successful execution rates (56% and 39%) due to the generation of more complex architectures that often fail validation.

Comparison with iterative LLM optimization methods. Our approach shares conceptual similarities with recent work on using LLMs as iterative optimizers [19, 25, 26], but differs in three critical dimensions that enable its effectiveness on neural architecture search:

Bounded Markovian memory. Unlike OPRO [25] or FunSearch [19], which populate prompts with historically global top-performing solutions (global elite retention), our sliding window maintains strictly the last $K{=}5$ sequential attempts. In code-level NAS, the architecture's structure evolves rapidly; therefore, ancient successes are less informative than immediate recent failures. The Markov property enforces that the LLM focuses entirely on local causal relationships (what modification just caused what specific error), avoiding the distraction of irrelevant global history.

Structured failure modeling. Our diagnostic triples explicitly encode code execution failures (tensor shape mismatches, `AttributeErrors` from undefined modules, syntax errors) as first-class entries in the history, enabling the LLM to learn from structural mistakes. This is fundamentally different from prior work where low-performing solutions are simply discarded from the prompt [19, 25] or managed implicitly through population selection [2]. As shown in Section 4, failure rates of 23–59% across our runs highlight that code generation for neural architectures is inherently error-prone; ignoring these failures would discard

Table 1. Comparison of the iterative NAS pipeline on CIFAR-10 (one-epoch proxy accuracy) across four LLMs. All models show significant improvement over single-shot generation.

Metric	**Qwen2.5-7B**	**Qwen3-4B**	**Qwen2.5-Coder**	**DeepSeek-Coder**
Total iterations	100	100	100	100
Successful evaluations	77	41	56	39
Success rate	77%	41%	56%	39%
Single-shot accuracy	0.380	0.100	0.464	0.377
Best accuracy (eval #)	0.593 (#68)	0.379 (#31)	0.636 (#36)	0.630 (#38)
First 5 evals (mean $\pm$ SD)	0.440 ± 0.062	0.134 ± 0.076	0.435 ± 0.081	0.377 ± 0.001
Last 5 evals (mean $\pm$ SD)	0.571 ± 0.032	0.371 ± 0.016	0.559 ± 0.125	0.590 ± 0.026
Improvement (last $-$ first 5)	$+0.131$	$+0.237$	$+0.124$	$+0.213$
Spearman ρ (p-value)	0.670 ($< 10^{-10}$)	0.787 ($< 10^{-9}$)	0.513 ($< 10^{-4}$)	0.556 ($< 10^{-3}$)
Slope (per 10 evals)	$+0.012$	$+0.065$	$+0.028$	$+0.055$
R^2	0.203	0.557	0.192	0.522

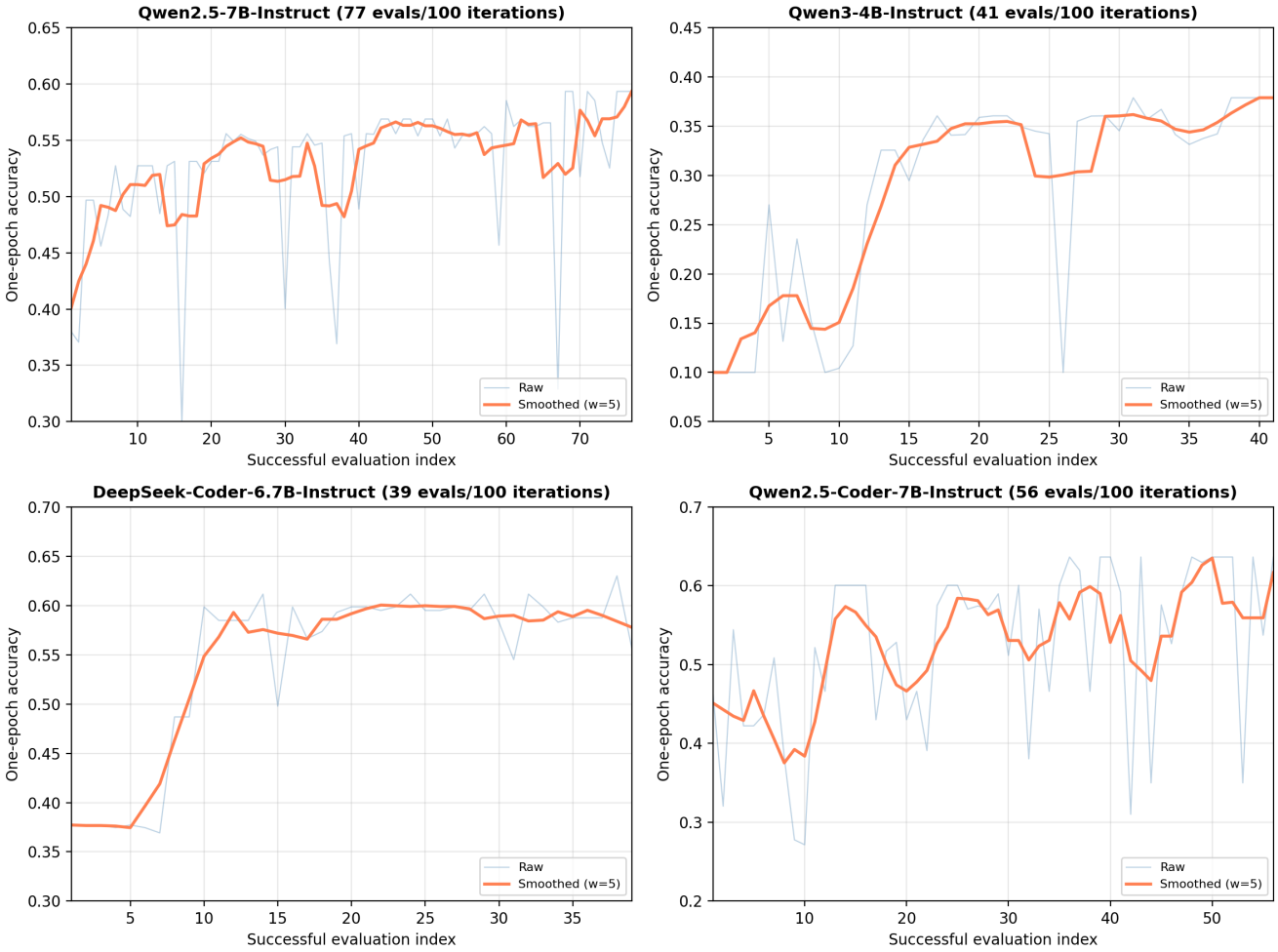

Figure 2. One-epoch proxy accuracy on CIFAR-10 across successful architecture evaluations for Qwen2.5-7B-Instruct (top-left, 77 evaluations), Qwen3-4B-Instruct (top-right, 41 evaluations), DeepSeek-Coder-6.7B-Instruct (bottom-left, 39 evaluations), and Qwen2.5-Coder-7B-Instruct (bottom-right, 56 evaluations). All models exhibit clear upward trends.

critical learning signals.

Small-model viability. Our results demonstrate that 4B-parameter LLMs can effectively perform NAS when equipped with structured feedback (Qwen3-4B: ρ=0.79,

Table 2. Systematic comparison with most similar LLM-based NAS and optimization methods. Our method uniquely combines open code space, bounded Markovian failure modeling, and small-model viability without population overhead.

Method	Target Domain	Search Space	Memory Structure	Markov Bound	Failure Modeling	Model Size
OPRO [25]	Text prompts, discrete	N/A	Top-K global elite	×	× Scores only	GPT-4, PaLM 2
FunSearch [19]	Math functions	Open Python	Multi-island pools	×	× Scores only	Codey (PaLM 2)
EvoPrompting [2]	Code-level NAS	Open code	Population diversity	×	× Selection only	Frozen / Soft-tuned
ReEvo [26]	Heuristic algorithms	Open code	Dual-layer reflection	×	✓ Verbal comparison	GPT-3.5 / GPT-4
LLMO [27]	NAS (CIFAR)	**Closed cell**	Historical score pairs	×	N/A (no code errors)	Gemini Pro (large)
Ours	**PyTorch NAS**	**Open code**	**K=5 sliding window**	✓	✓ **Diagnostic triples**	**4B/7B frozen**

$p<10^{-9}$), whereas prior iterative LLM optimization work primarily relies on large models (OPRO uses GPT-4 and PaLM 2-L [25]). We hypothesize this is enabled by our dual-LLM specialization: the Code Generator focuses solely on code synthesis given structured suggestions, while the Prompt Improver handles diagnostic reasoning. This task decomposition reduces per-call cognitive load compared to end-to-end generation [2].

Table 2 positions our method relative to the five most similar LLM-based NAS and optimization approaches. We are the **only method** that simultaneously satisfies three critical properties: (1) open code space with structural constraints, (2) bounded Markovian memory with explicit failure handling, and (3) demonstrated effectiveness on small ($\leq$7B) instruction-tuned LLMs without additional training. Notably, while LLMO also applies LLMs to NAS without training, it operates in predefined discrete search spaces that fundamentally limit architectural expressiveness.

One-epoch proxy as architecture ranking signal. Using one-epoch accuracy as a ranking signal offers a pragmatic trade-off between evaluation cost and informativeness. While the absolute accuracy values (10–59%) are well below state-of-the-art CIFAR-10 results with full training (exceeding 95%), prior work on training-free NAS [13, 15] has established that cheap proxy metrics can effectively rank architectures. Our one-epoch proxy serves a similar role: it distinguishes between architectures of varying quality at minimal cost, enabling the feedback loop to guide the LLM toward better designs. Retraining the best-discovered architectures with full training schedules could verify whether the one-epoch ranking transfers to converged performance.

Limitations. Our study has several limitations. First, although we compare two LLMs of different sizes, each is evaluated with a single run of 100 iterations; multi-run experiments would strengthen statistical confidence. Second, the one-epoch proxy may not perfectly correlate with full-training performance; retraining top architectures to convergence is needed to assess their true potential. Third, the code generation success rates (41–77%) indicate that current LLMs still struggle with structurally valid neural network code, suggesting opportunities for improved prompting strategies or constrained decoding. Finally, our evaluation is limited to CIFAR-10; experiments on larger-scale benchmarks and diverse tasks would better demonstrate versatility.

Future work. Promising directions include: (1) retraining the top-K discovered architectures with full training schedules to verify their converged performance; (2) extending the cross-model comparison to additional LLM families and sizes; (3) integrating zero-shot proxy metrics [13, 15] as complementary evaluation signals to further accelerate the search; (4) extending to larger datasets (e.g., ImageNet) and tasks beyond image classification; and (5) apply genetic algorithm for historical feedback memory.

6. Conclusion

We presented an iterative NAS pipeline that leverages instruction-tuned LLMs to progressively generate and improve neural network architectures through a closed loop of code generation, evaluation, and prompt refinement with historical feedback memory. Evaluated on CIFAR-10 with four LLMs of different sizes and specializations, the pipeline significantly improved one-epoch proxy accuracy across all models (e.g., from 38.0% to 59.3% with Qwen2.5-7B, $\rho = 0.67$; and peak accuracies up to 63.6% with code models), demonstrating that iterative feedback with bounded historical memory significantly outperforms single-shot generation across model sizes. Three design principles distinguish our approach from prior iterative LLM optimization methods: (1) operating in an open code space rather than constrained predefined cells, (2) bounded K-step Markovian memory prevents context overflow while retaining relevant failure patterns, and (3) structured diagnostic triples explicitly model code execution failures, which alongside dual-LLM specialization (Code Generator and Prompt Improver) enables effective operation on small (4B) frozen models without fine-tuning. Our results establish that even small LLMs can serve as effective architecture search agents when equipped with structured iterative

feedback, offering a lightweight and accessible approach to neural architecture search.

References

[1] Nada Aboudeshish, Dmitry Ignatov, and Radu Timofte. Augmentgest: Can random data cropping augmentation boost gesture recognition performance? *arXiv preprint*, arXiv:2506.07216, 2025. 3

[2] Angelica Chen, David Dohan, and Daved Hutchins. Evoprompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, 2023. 1, 2, 5, 7

[3] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint*, arXiv:2107.03374, 2021. 2

[4] Saif U Din, Muhammad Ahsan Hussain, Mohsin Ikram, Dmitry Ignatov, and Radu Timofte. Ai on the edge: An automated pipeline for pytorch-to-android deployment and benchmarking. *Preprints*, 2025. 2

[5] Mohamed Gado, Towhid Taliee, Muhammad Danish Memon, Dmitry Ignatov, and Radu Timofte. Vist-gpt: Ushering in the era of visual storytelling with llms? *arXiv preprint*, arXiv:2504.19267, 2025. 2

[6] Arash Torabi Goodarzi, Roman Kochnev, Waleed Khalid, Furui Qin, Tolgay Atinc Uzun, Yashkumar Sanjaybhai Dhameliya, Yash Kanubhai Kathiriya, Zofia Antonina Bentyn, Dmitry Ignatov, and Radu Timofte. LEMUR Neural Network Dataset: Towards Seamless AutoML. *arXiv preprint*, arXiv:2504.10552, 2025. 2

[7] Krunal Jesani, Dmitry Ignatov, and Radu Timofte. Llm as a neural architect: Controlled generation of image captioning models under strict api contracts. *arXiv preprint*, arXiv:2512.14706, 2025. 1, 2

[8] Waleed Khalid, Dmitry Ignatov, and Radu Timofte. A retrieval-augmented generation approach to extracting algorithmic logic from neural networks. *arXiv preprint*, arXiv:2512.04329, 2025. 2

[9] Waleed Khalid, Dmitry Ignatov, and Radu Timofte. From memorization to creativity: Llm as a designer of novel neural-architectures. *arXiv preprint*, arXiv:2601.02997, 2026. 1, 2

[10] Roman Kochnev, Arash Torabi Goodarzi, Zofia Antonina Bentyn, Dmitry Ignatov, and Radu Timofte. Optuna vs Code Llama: Are LLMs a New Paradigm for Hyperparameter Tuning? In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 5664–5674, 2025. 2

[11] Roman Kochnev, Waleed Khalid, Tolgay Atinc Uzun, Xi Zhang, Yashkumar Sanjaybhai Dhameliya, Furui Qin, Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Dmitry Ignatov, and Radu Timofte. Nngpt: Rethinking automl with large language models. *arXiv preprint*, arXiv:2511.2033, 2025. 1, 2

[12] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. 1, 3

[13] Guihong Li et al. Zero-shot neural architecture search: Challenges, solutions, and opportunities. *IEEE TPAMI*, 46(6): 4071–4088, 2024. 1, 2, 7

[14] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *ICLR*, 2019. 1, 2

[15] Joseph Mellor, Jack Turner, Amos Shersby, and Elliot J. Shersby. Neural architecture search without training. In *ICML*, pages 7588–7598, 2021. 1, 2, 3, 7

[16] Yash Mittal, Dmitry Ignatov, and Radu Timofte. Preparation of fractal-inspired computational architectures for advanced large language model analysis. *arXiv preprint*, arXiv:2511.07329, 2025. 1, 2

[17] Hieu Pham, Melody Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *ICML*, pages 4095–4104, 2018. 2

[18] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *AAAI*, pages 4780–4789, 2019. 1, 2

[19] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625 (7995):468–475, 2023. 1, 2, 3, 5, 7

[20] Bhavya Rupani, Dmitry Ignatov, and Radu Timofte. Exploring the collaboration between vision models and llms for enhanced image classification. *Preprints*, 2025. 2

[21] Usha Shrestha, Dmitry Ignatov, and Radu Timofte. From brute force to semantic insight: Performance-guided data transformation design with llms. *arXiv preprint*, arXiv:2601.03808, 2026. 2

[22] Tolgay Atinc Uzun, Dmitry Ignatov, and Radu Timofte. Closed-loop llm discovery of non-standard channel priors in vision models. *arXiv preprint*, arXiv:2601.08517, 2026. 1, 2

[23] Tolgay Atincand Uzun, Waleed Khalid, Saif U Din, Sai Revanth Mulukuledu, Akashdeep Singh, Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Yashkumar Rajeshbhai Lukhi, Ahsan Hussain, Krunal Jesani, Usha Shrestha, Yash Mittal, Roman Kochnev, Pritam Kadam, Mohsin Ikram, Harsh Rameshbhai Moradiya, Alice Arslanian, Dmitry Ignatov, and Radu Timofte. Lemur 2: Unlocking neural network diversity for ai. *arXiv preprint*, 2026. 2

[24] Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Dmitry Ignatov, and Radu Timofte. Enhancing llm-based neural network generation: Few-shot prompting and efficient validation for automated architecture design. *arXiv preprint*, arXiv:2512.24120, 2025. 1, 2

[25] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*, 2023. 1, 2, 3, 5, 7

[26] Fei Ye, Haoran Yang, Xiaohan Cai, et al. Reevo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, 2024. 2, 5, 7

[27] Yuhang Zhong, Yuhao Liu, Xiaotong Liu, and Tao Chen. Large language model assisted adversarial robustness neural architecture search. *IEEE Transactions on Artificial Intelligence*, 2024. 1, 2, 7

[28] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *ICLR*, 2017. 1, 2